

Escape From Heidelberg Dynamics

Generated by Doxygen 1.12.0

1 Escape from Heidelberg Dynamics Game Systems	1
2 Coding Style	3
2.1 Example	3
3 Development	5
3.1 Submodules on Windows for use in <code>master</code>	5
3.2 Developing & Testing the Branch on its own, without <code>godot</code> and <code>master</code>	5
3.2.1 Using Nix	5
3.2.2 On Ubuntu	5
3.2.3 On Windows	6
3.2.4 Running Tests	6
3.3 Doxygen	6
3.3.1 Windows	6
3.3.2 Ubuntu	6
4 Project Architecture	7
5 Namespace Index	9
5.1 Package List	9
6 Class Index	11
6.1 Class List	11
7 Namespace Documentation	13
7.1 TestProject1 Namespace Reference	13
8 Class Documentation	15
8.1 Health Class Reference	15
8.1.1 Detailed Description	16
8.1.2 Constructor & Destructor Documentation	16
8.1.2.1 Health()	16
8.1.3 Member Function Documentation	17
8.1.3.1 ApplyDamage()	17
8.1.3.2 ApplyTrueDamage()	17
8.1.3.3 damageHP()	17
8.1.3.4 damageShield()	17
8.1.3.5 HealBoth()	17
8.1.3.6 HealHP()	18
8.1.3.7 HealShield()	18
8.2 TestProject1.UnitTest1 Class Reference	18
8.2.1 Detailed Description	19
Index	21

Chapter 1

Escape from Heidelberg Dynamics Game Systems



Figure 1.1 our protagonist

This project includes the health system implemented in C#, tested using XUnit.

You may want to read:

- [Coding Style](#)

- [Developer Information](#)
- [Project Architecture](#)

Chapter 2

Coding Style

The Coding Style is based on the [MS .NET C# coding style](#):

Class Names and Namespaces are named in PascalCase.

Functions, Parameters, and Variables of any kind are to be named in camelCase.

Both public and private variables are to be prefixed with an underscore.

2.1 Example

```
using System;

public class Health
{
    private int _hp;

    // Constructor to initialize health with a starting value
    public Health(int initialHp)
    {
        _hp = initialHp;
    }

    // Function to apply damage, using Math.Max to ensure hp doesn't go below 0
    public void Damage(int amount)
    {
        _hp = Math.Max(0, _hp - amount);
    }

    // Function to apply healing, using Math.Min to ensure hp doesn't exceed a maximum value (e.g., 100)
    public void Heal(int amount)
    {
        int maxHealth = 100; // You can modify this value based on your requirements
        _hp = Math.Min(maxHealth, _hp + amount);
    }

    // Function to get the current health value
    public int GetHealth()
    {
        return _hp;
    }
}
```


Chapter 3

Development

This file contains all the necessary information to set up your development environment and work on the game systems.

3.1 Submodules on Windows for use in master

GitHub Desktop on Windows doesn't play nice with Git Submodules. To work around this, we use Git on PowerShell. Most comfortably done through [Windows Terminal](#).

```
cd .\Documents\GitHub\  
git clone --recurse-submodules -j8 https://github.com/reogaro/heidelberg-dynamics.git
```

To update the submodule, use `git submodule update --remote`.

3.2 Developing & Testing the Branch on its own, without godot and master

Development work on this branch can be done without godot or the master branch.

3.2.1 Using Nix

1. **Install Nix:** Follow the instructions at <https://nixos.org/download.html>.
2. **Enter Nix Shell:** Run the following command in the project directory to enter the Nix environment with Godot 4 and .NET SDK 8.
`nix-shell`

3.2.2 On Ubuntu

If you prefer to set up .NET SDK 8 directly on Ubuntu without using Nix:

1. **Install the Microsoft package repository and GPG keys:**

```
wget https://packages.microsoft.com/config/ubuntu/20.04/packages-microsoft-prod.deb -O  
packages-microsoft-prod.deb  
sudo dpkg -i packages-microsoft-prod.deb  
sudo apt-get update  
sudo apt-get install -y dotnet-sdk-8.0
```

3.2.3 On Windows

- Install VSCode
- Install `.NET 8.0 SDK` for VSCode
- Use PowerShell and `dotnet.exe` instead of `bash` and `dotnet`

Or just use `WSL`

3.2.4 Running Tests

To run the XUnit tests in the `TestProject1` directory, use the following command:

```
dotnet test TestProject1.sln
```

This will build the project and execute all tests, providing a summary of the test results.

3.3 Doxygen

This Project uses Doxygen for Documentation. Just run `doxygen` in this projects root directory and the documentation will be generated in the `docs/doxygen` folder.

Creating PDFs from the LaTeX output also requires `pdflatex`.

3.3.1 Windows

- `Download` & install Doxygen
- Open a new PowerShell instance
- `cd` to the `heidelberg-dynamics` folder
- run `doxygen.exe`
- open `docs/doxygen/html/index.html` in your browser

For PDF Generation

- Install `TeX Live` for Windows
- Update TeX Live Packages
- Reboot your PC (seriously)
- run `docs/doxygen/latex/build.bat`
- open `docs/doxygen/latex/refman.pdf`

3.3.2 Ubuntu

- Install Doxygen using `sudo apt install doxygen`
- Open a new Terminal
- `cd` to the `heidelberg-dynamics` folder
- run `doxygen`
- open `docs/doxygen/html/index.html` in your browser

Chapter 4

Project Architecture

One of the given fixed requirements was to write C# code and use Doxygen or Sphinx for automated documentation generation. This approach is sadly *fundamentally incompatible* with just adding doxygen to the Godot project.

To work around those limitations and still fulfill the documentation requirements, we separated the parts that are easy to test and document into a *seperate git branch*, which is then embedded in the `master` branch using git submodules.

Another advantage of this approach is that it frees us from godots limited testing capabilities and allows for using a more "open" testing framework like XUnit.

GitHub Desktop on Windows doesn't play well with Git Submodules, read the [related article](#) for alternatives.

Chapter 5

Namespace Index

5.1 Package List

Here are the packages with brief descriptions (if available):

TestProject1	13
--	----

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Health	This class models the HP of an entity, health, shield, resistances, and vulnerabilities	15
TestProject1.UnitTest1	Unit Tests for Health.cs	18

Chapter 7

Namespace Documentation

7.1 TestProject1 Namespace Reference

Classes

- class [UnitTest1](#)

Unit Tests for Health.cs.

Chapter 8

Class Documentation

8.1 Health Class Reference

This class models the HP of an entity, health, shield, resistances, and vulnerabilities.

Public Member Functions

- [Health](#) (int hpMax, int shieldMax, int initialResistancePhysical, int initialResistanceElectric, int vulnPhysical, int vulnElectric)
Initializes a new instance of the [Health](#) class.
- void [ApplyTrueDamage](#) (int damage)
Applies true damage to shield and HP, ignoring resistances.
- void [ApplyDamage](#) (int damagePhysical, int damageElectric)
Applies physical and electric damage, taking into account resistances and vulnerabilities.
- void [HealHP](#) (int amount)
Heals a specified amount of HP.
- void [HealShield](#) (int amount)
Heals a specified amount of shield.
- void [HealBoth](#) (int healAmount)
Heals both HP and shield, prioritizing HP.
- void **DestroyShield** ()
Completely destroys the shield.
- void **RestoreHealth** ()
Restores HP to maximum.
- void **RestoreShield** ()
Restores shield to maximum.
- void [damageShield](#) (int damage)
Damages the shield by a specified amount.
- void [damageHP](#) (int damage)
Damages HP by a specified amount.
- void **Revive** ()
Revives the entity with 1 HP and no shield.

Static Public Attributes

- `const int _damageBias = 12`
Damage bias applied to all damage calculations.

Properties

- `int _hp` [get]
Current hit points.
- `int _hpMax` [get]
Maximum hit points.
- `int _shield` [get]
Current shield points.
- `int _shieldMax` [get]
Maximum shield points.
- `int _resistancePhysical` [get]
Resistance against physical damage.
- `int _resistanceElectric` [get]
Resistance against electric damage.
- `int _vulnPhysical` [get]
Vulnerability to physical damage.
- `int _vulnElectric` [get]
Vulnerability to electric damage.
- `bool _alive = true` [get]
Indicates whether the entity is alive.

8.1.1 Detailed Description

This class models the HP of an entity, health, shield, resistances, and vulnerabilities.

8.1.2 Constructor & Destructor Documentation

8.1.2.1 Health()

```
Health.Health (
    int hpMax,
    int shieldMax,
    int initialResistancePhysical,
    int initialResistanceElectric,
    int vulnPhysical,
    int vulnElectric)
```

Initializes a new instance of the [Health](#) class.

Parameters

<i>hpMax</i>	Maximum hit points.
<i>shieldMax</i>	Maximum shield points.
<i>initialResistancePhysical</i>	Initial physical resistance.
<i>initialResistanceElectric</i>	Initial electric resistance.
<i>vulnPhysical</i>	Physical vulnerability.
<i>vulnElectric</i>	Electric vulnerability.

8.1.3 Member Function Documentation

8.1.3.1 ApplyDamage()

```
void Health.ApplyDamage (  
    int damagePhysical,  
    int damageElectric)
```

Applies physical and electric damage, taking into account resistances and vulnerabilities.

Parameters

<i>damagePhysical</i>	Physical damage amount.
<i>damageElectric</i>	Electric damage amount.

8.1.3.2 ApplyTrueDamage()

```
void Health.ApplyTrueDamage (  
    int damage)
```

Applies true damage to shield and HP, ignoring resistances.

Parameters

<i>damage</i>	The amount of true damage to apply.
---------------	-------------------------------------

8.1.3.3 damageHP()

```
void Health.damageHP (  
    int damage)
```

Damages HP by a specified amount.

Parameters

<i>damage</i>	Amount of damage to apply to HP.
---------------	----------------------------------

8.1.3.4 damageShield()

```
void Health.damageShield (  
    int damage)
```

Damages the shield by a specified amount.

Parameters

<i>damage</i>	Amount of damage to apply to the shield.
---------------	--

8.1.3.5 HealBoth()

```
void Health.HealBoth (  
    int healAmount)
```

Heals both HP and shield, prioritizing HP.

Parameters

<i>healAmount</i>	Total amount to heal.
-------------------	-----------------------

8.1.3.6 HealHP()

```
void Health.HealHP (
    int amount)
```

Heals a specified amount of HP.

Parameters

<i>amount</i>	Amount of HP to heal.
---------------	-----------------------

8.1.3.7 HealShield()

```
void Health.HealShield (
    int amount)
```

Heals a specified amount of shield.

Parameters

<i>amount</i>	Amount of shield to heal.
---------------	---------------------------

The documentation for this class was generated from the following file:

- TestProject1/Health.cs

8.2 TestProject1.UnitTest1 Class Reference

Unit Tests for Health.cs.

Public Member Functions

- **UnitTest1** (ITestOutputHelper testLogger)
- void **Health_Initialization_Test** ()
Tests the constructor.
- void **True_Damage_Test** (int startHP, int startShield, int appliedDamage, int expectedHp, int expectedShield, bool expectedAlive)
Tests ApplyTrueDamage()
- void **ApplyDamage_Test** (int initialResistancePhysical, int initialResistanceElectric, int initialVulnPhysical, int initialVulnElectric, int damagePhysical, int damageElectric, int expectedHp, int expectedShield, bool expectedAlive)
Tests ApplyDamage()
- void **HealBoth_Test** (int initialHP, int initialShield, int damageHP, int damageShield, int healAmount, int expectedHp, int expectedShield, bool expectedAlive)
Tests HealBoth()

8.2.1 Detailed Description

Unit Tests for Health.cs.

The documentation for this class was generated from the following file:

- TestProject1/UnitTest1.cs

Index

ApplyDamage
 Health, [17](#)
ApplyTrueDamage
 Health, [17](#)

Coding Style, [3](#)

damageHP
 Health, [17](#)
damageShield
 Health, [17](#)
Development, [5](#)

Escape from Heidelberg Dynamics Game Systems, [1](#)

HealBoth
 Health, [17](#)
HealHP
 Health, [18](#)
HealShield
 Health, [18](#)
Health, [15](#)
 ApplyDamage, [17](#)
 ApplyTrueDamage, [17](#)
 damageHP, [17](#)
 damageShield, [17](#)
 HealBoth, [17](#)
 HealHP, [18](#)
 HealShield, [18](#)
 Health, [16](#)

Project Architecture, [7](#)

TestProject1, [13](#)
TestProject1.UnitTest1, [18](#)